Assignment 4 – Multi-Token Ecosystem Project Report

Aimaut, Bekdaulet

Our GitHub

PART 1 –

Token Implementation & Deployment

Task 1: ERC-20 Token Implementation

Contract Overview

The TOKEN →

contracts > Token4.sol > Token4

```sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import "@openzeppelin/contracts/access/Ownable.sol";

contract Token4 is ERC20, Ownable {
    constructor()
        ERC20("Token4", "TK4")
        Ownable(msg.sender)
    {
        _mint(msg.sender, 1_000_000 * 10 ** decimals());
    }

    function mint(address to, uint256 amount) external onlyOwner {
        _mint(to, amount);
    }
}
```

The deploy script →

scripts > deployToken4.js > ...

```js
const hre = require("hardhat");

async function main() {
  console.log("Starting deployment...");

  const Token4 = await hre.ethers.getContractFactory("Token4");
  const token = await Token4.deploy();

  await token.waitForDeployment();

  console.log("Token4 deployed to:", await token.getAddress());
}

main().catch((error) => {
  console.error(error);
  process.exitCode = 1;
});
```

Token Name: Token4 Token

Symbol: TK4

Total Supply: 1,000,000

TK4 Deployment Network: Hardhat Local Network

Implementation Details

- The ERC-20 token was implemented using the OpenZeppelin ERC20 standard.
- The contract inherits from ERC20 and Ownable.
- Initial supply is minted to the deployer, and an additional mint() function is implemented with onlyOwner restriction.

Key Features Implemented

✓ transfer()

✓ approve()

✓ transferFrom()

✓ allowance()

✓ Transfer & Approval events

✓ Minting logic restricted to owner

Deployment Information:

- **Contract Address:** 0x5FbDB2315678afecb367f032d93F642f64180aa3
- **Deployment Network:** Hardhat Local Network
- **Deployment Tool:** Hardhat + Ethers.js
- **Deployer:** First Hardhat account
- **Gas Used:** Shown in Hardhat console output

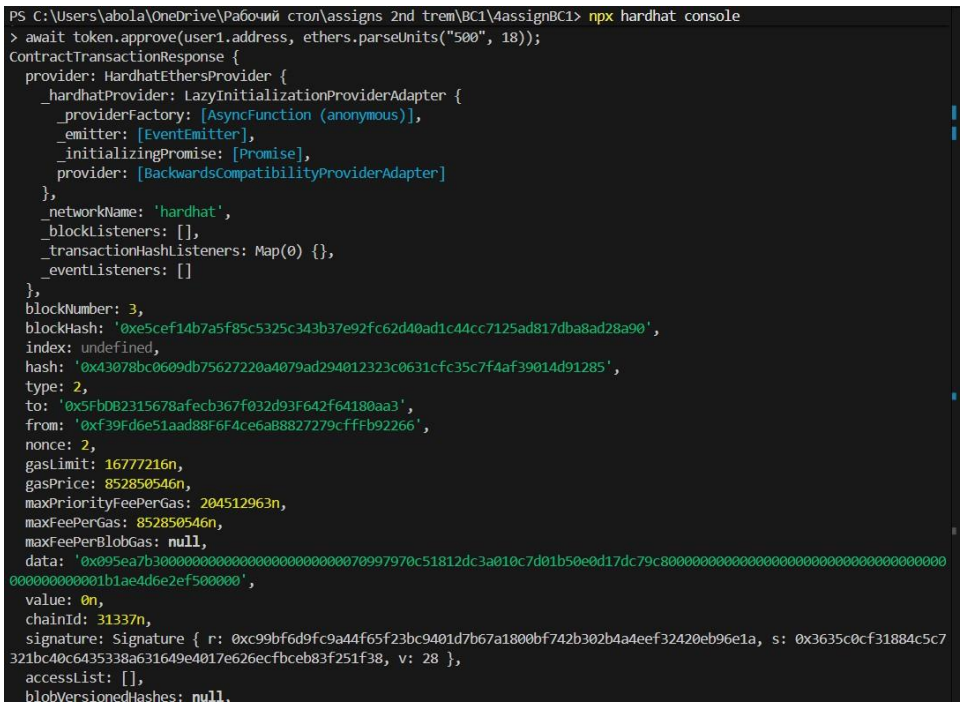
For tests I have deployed token in hardhat console

```
>
> const Token4 = await ethers.getContractFactory("Token4");
undefined
> const token = await Token4.deploy();
undefined
> await token.waitForDeployment();
BaseContract {
  target: '0x5FbDB2315678afecb367f032d93F642f64180aa3',
  interface: Interface {
    fragments: [
      [ConstructorFragment], [ErrorFragment],
      [ErrorFragment],       [ErrorFragment],
      [ErrorFragment],       [ErrorFragment],
      [ErrorFragment],       [ErrorFragment],
      [ErrorFragment],       [EventFragment],
      [EventFragment],       [EventFragment],
      [FunctionFragment],    [FunctionFragment],
      [FunctionFragment],    [FunctionFragment],
      [FunctionFragment],    [FunctionFragment],
      [FunctionFragment],    [FunctionFragment],
      [FunctionFragment],    [FunctionFragment],
      [FunctionFragment],    [FunctionFragment],
      [FunctionFragment]
    ],
    deploy: ConstructorFragment {
      type: 'constructor',
      inputs: [],
      payable: false,
      gas: null
    },
    fallback: null,
    receive: false
  },
```

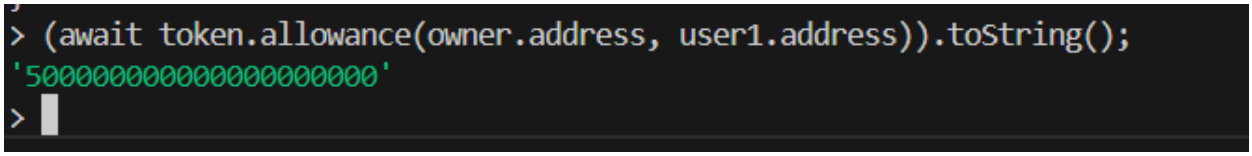
transfer through hardhat console

```
PS C:\Users\abola\OneDrive\Рабочий стол\assigns 2nd trem\BC1\4assignBC1> npx hardhat console
> await token.transfer(user1.address, ethers.parseUnits("1000", 18));
ContractTransactionResponse {
  provider: HardhatEthersProvider {
    _hardhatProvider: LazyInitializationProviderAdapter {
      _providerFactory: [AsyncFunction (anonymous)],
      _emitter: [EventEmitter],
      _initializingPromise: [Promise],
      provider: [BackwardsCompatibilityProviderAdapter]
    },
    _networkName: 'hardhat',
    _blockListeners: [],
    _transactionHashListeners: Map(0) {},
    _eventListeners: []
  },
  blockNumber: 2,
  blockHash: '0xc665c1a733ce7178dfe6b657ab0991df4a6b6125d2c4265d247b6783bdf3c7ac',
  index: undefined,
  hash: '0x982fdae230d318c6b7b2aa25630d7fe40ed24661852a9751da16e44f48bd201e',
  type: 2,
  to: '0x5FbDB2315678afecb367f032d93F642f64180aa3',
  from: '0xf39Fd6e51aad88F6F4ce6aB8827279cfffb92266',
  nonce: 1,
  gasLimit: 16777216n,
  gasPrice: 974444119n,
  maxPriorityFeePerGas: 232421875n,
  maxFeePerGas: 974444119n,
  maxFeePerBlobGas: null,
  data: '0xa9059cbb0000000000000000000000007099790c51812dc3a010c7d01b50e0d17dc79c800000000000000000000000000000000000000
00000000003635c9adc5dea00000',
  value: 0n,
  chainId: 31337n,
  signature: Signature { r: 0xc28b8215b3026b664b2de80e25bc4ef448d09f0a3edab28881a50629c2bca306, s: 0x2f770f63a19a1e8ce
06aea3c7fbc561add981907c87da20737af5cbd126178bd, v: 28 },
  accessList: [],
  blobVersionedHashes: null,
```

approve through hardhat console\



Allowance check



Here is allowance > 0

transferFrom through hardhat console

```
> await token.connect(user1).transferFrom(
|   owner.address,
|   user2.address,
|   ethers.parseUnits("200", 18)
| );
ContractTransactionResponse {
  provider: HardhatEthersProvider {
    _hardhatProvider: LazyInitializationProviderAdapter {
      _providerFactory: [AsyncFunction (anonymous)],
      _emitter: [EventEmitter],
      _initializingPromise: [Promise],
      provider: [BackwardsCompatibilityProviderAdapter]
    },
    _networkName: 'hardhat',
    _blockListeners: [],
    _transactionHashListeners: Map(0) {},
    _eventListeners: []
  },
  blockNumber: 4,
  blockHash: '0xd8c75fab42e1aa0e68131397e6f6e60b6bfabf580c612aaea6ec9c61fe30ee27',
  index: undefined,
  hash: '0x485604e8c0eb752400c5238958c7544a5d5f3797b7ea2ead0715442dcb3545d8',
  type: 2,
  to: '0x5FbDB2315678afecb367f032d93F642f64180aa3',
  from: '0x70997970C51812dc3A010C7d01b50e0d17dc79C8',
  nonce: 0,
  gasLimit: 16777216n,
  gasPrice: 746411117n,
  maxPriorityFeePerGas: 178993324n,
  maxFeePerGas: 746411117n,
  maxFeePerBlobGas: null,
```

balances through hardhat console

```
> (await token.balanceOf(owner.address)).toString();
'998800000000000000000000'
> (await token.balanceOf(user1.address)).toString();
'100000000000000000000'
> (await token.balanceOf(user2.address)).toString();
'200000000000000000000'
>
```

The contract was deployed on the Hardhat local network.
All interactions were performed within the same EVM session to ensure consistent state.

Contract ABI

```
"abi": [
    {
      "inputs": [],
      "stateMutability": "nonpayable",
      "type": "constructor"
    },
    {
      "inputs": [
        {
          "internalType": "address",
          "name": "spender",
          "type": "address"
```

```
      },
      {
        "internalType": "uint256",
        "name": "allowance",
        "type": "uint256"
      },
      {
        "internalType": "uint256",
        "name": "needed",
        "type": "uint256"
      }
    ],
    "name": "ERC20InsufficientAllowance",
    "type": "error"
  },
  {
    "inputs": [
      {
        "internalType": "address",
        "name": "sender",
        "type": "address"
      },
      {
        "internalType": "uint256",
        "name": "balance",
        "type": "uint256"
      },
      {
        "internalType": "uint256",
        "name": "needed",
        "type": "uint256"
      }
    ],
    "name": "ERC20InsufficientBalance",
    "type": "error"
  },
  {
    "inputs": [
      {
        "internalType": "address",
        "name": "approver",
        "type": "address"
      }
    ],
    "name": "ERC20InvalidApprover",
    "type": "error"
  },
  {
    "inputs": [
      {
        "internalType": "address",
```

```
        "name": "receiver",
        "type": "address"
      }
    ],
    "name": "ERC20InvalidReceiver",
    "type": "error"
  },
  {
    "inputs": [
      {
        "internalType": "address",
        "name": "sender",
        "type": "address"
      }
    ],
    "name": "ERC20InvalidSender",
    "type": "error"
  },
  {
    "inputs": [
      {
        "internalType": "address",
        "name": "spender",
        "type": "address"
      }
    ],
    "name": "ERC20InvalidSpender",
    "type": "error"
  },
  {
    "inputs": [
      {
        "internalType": "address",
        "name": "owner",
        "type": "address"
      }
    ],
    "name": "OwnableInvalidOwner",
    "type": "error"
  },
  {
    "inputs": [
      {
        "internalType": "address",
        "name": "account",
        "type": "address"
      }
    ],
    "name": "OwnableUnauthorizedAccount",
    "type": "error"
  },
```

```
  {
    "anonymous": false,
    "inputs": [
      {
        "indexed": true,
        "internalType": "address",
        "name": "owner",
        "type": "address"
      },
      {
        "indexed": true,
        "internalType": "address",
        "name": "spender",
        "type": "address"
      },
      {
        "indexed": false,
        "internalType": "uint256",
        "name": "value",
        "type": "uint256"
      }
    ],
    "name": "Approval",
    "type": "event"
  },
  {
    "anonymous": false,
    "inputs": [
      {
        "indexed": true,
        "internalType": "address",
        "name": "previousOwner",
        "type": "address"
      },
      {
        "indexed": true,
        "internalType": "address",
        "name": "newOwner",
        "type": "address"
      }
    ],
    "name": "OwnershipTransferred",
    "type": "event"
  },
  {
    "anonymous": false,
    "inputs": [
      {
        "indexed": true,
        "internalType": "address",
        "name": "from",
```

```
        "type": "address"
      },
      {
        "indexed": true,
        "internalType": "address",
        "name": "to",
        "type": "address"
      },
      {
        "indexed": false,
        "internalType": "uint256",
        "name": "value",
        "type": "uint256"
      }
    ],
    "name": "Transfer",
    "type": "event"
  },
  {
    "inputs": [
      {
        "internalType": "address",
        "name": "owner",
        "type": "address"
      },
      {
        "internalType": "address",
        "name": "spender",
        "type": "address"
      }
    ],
    "name": "allowance",
    "outputs": [
      {
        "internalType": "uint256",
        "name": "",
        "type": "uint256"
      }
    ],
    "stateMutability": "view",
    "type": "function"
  },
  {
    "inputs": [
      {
        "internalType": "address",
        "name": "spender",
        "type": "address"
      },
      {
        "internalType": "uint256",
```

```
        "name": "value",
        "type": "uint256"
      }
    ],
    "name": "approve",
    "outputs": [
      {
        "internalType": "bool",
        "name": "",
        "type": "bool"
      }
    ],
    "stateMutability": "nonpayable",
    "type": "function"
  },
  {
    "inputs": [
      {
        "internalType": "address",
        "name": "account",
        "type": "address"
      }
    ],
    "name": "balanceOf",
    "outputs": [
      {
        "internalType": "uint256",
        "name": "",
        "type": "uint256"
      }
    ],
    "stateMutability": "view",
    "type": "function"
  },
  {
    "inputs": [],
    "name": "decimals",
    "outputs": [
      {
        "internalType": "uint8",
        "name": "",
        "type": "uint8"
      }
    ],
    "stateMutability": "view",
    "type": "function"
  },
  {
    "inputs": [
      {
        "internalType": "address",
```

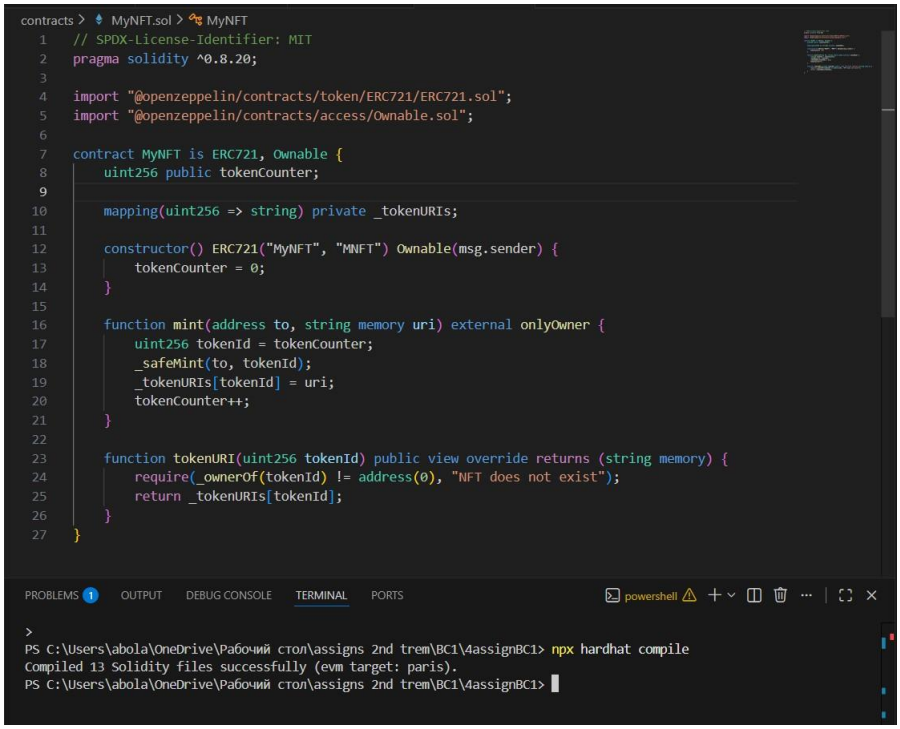
```
        "name": "to",
        "type": "address"
      },
      {
        "internalType": "uint256",
        "name": "amount",
        "type": "uint256"
      }
    ],
    "name": "mint",
    "outputs": [],
    "stateMutability": "nonpayable",
    "type": "function"
  },
  {
    "inputs": [],
    "name": "name",
    "outputs": [
      {
        "internalType": "string",
        "name": "",
        "type": "string"
      }
    ],
    "stateMutability": "view",
    "type": "function"
  },
  {
    "inputs": [],
    "name": "owner",
    "outputs": [
      {
        "internalType": "address",
        "name": "",
        "type": "address"
      }
    ],
    "stateMutability": "view",
    "type": "function"
  },
  {
    "inputs": [],
    "name": "renounceOwnership",
    "outputs": [],
    "stateMutability": "nonpayable",
    "type": "function"
  },
  {
    "inputs": [],
    "name": "symbol",
    "outputs": [
```

```
      {
        "internalType": "string",
        "name": "",
        "type": "string"
      }
    ],
    "stateMutability": "view",
    "type": "function"
  },
  {
    "inputs": [],
    "name": "totalSupply",
    "outputs": [
      {
        "internalType": "uint256",
        "name": "",
        "type": "uint256"
      }
    ],
    "stateMutability": "view",
    "type": "function"
  },
  {
    "inputs": [
      {
        "internalType": "address",
        "name": "to",
        "type": "address"
      },
      {
        "internalType": "uint256",
        "name": "value",
        "type": "uint256"
      }
    ],
    "name": "transfer",
    "outputs": [
      {
        "internalType": "bool",
        "name": "",
        "type": "bool"
      }
    ],
    "stateMutability": "nonpayable",
    "type": "function"
  },
  {
    "inputs": [
      {
        "internalType": "address",
        "name": "from",
```

```
        "type": "address"
      },
      {
        "internalType": "address",
        "name": "to",
        "type": "address"
      },
      {
        "internalType": "uint256",
        "name": "value",
        "type": "uint256"
      }
    ],
    "name": "transferFrom",
    "outputs": [
      {
        "internalType": "bool",
        "name": "",
        "type": "bool"
      }
    ],
    "stateMutability": "nonpayable",
    "type": "function"
  },
  {
    "inputs": [
      {
        "internalType": "address",
        "name": "newOwner",
        "type": "address"
      }
    ],
    "name": "transferOwnership",
    "outputs": [],
    "stateMutability":
    "nonpayable", "type":
    "function"
  }
```

Task 2: ERC-721 NFT Contract Implementation

Contract Overview



- **NFT Collection Name:** MyNFT
- **Symbol:** MNFT
- **Total Minted:** 3 NFTs
- **Deployment Network:** Hardhat Local Network

NFT Metadata

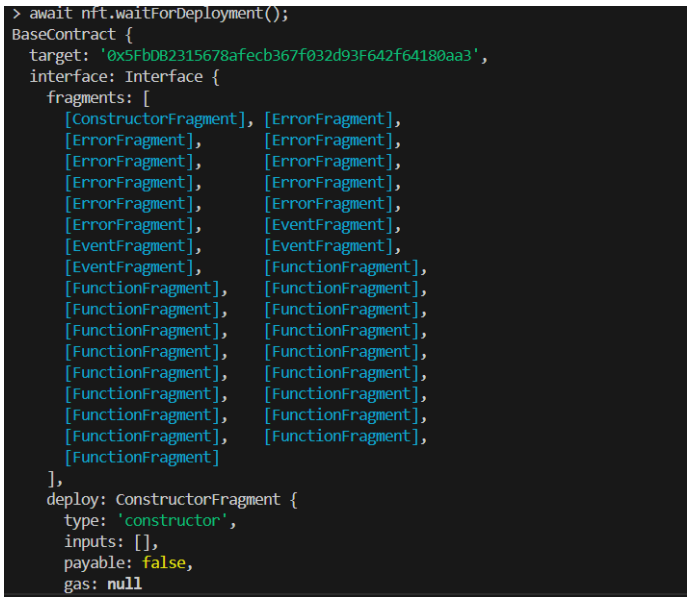
- **Token ID #0:** ipfs://nft1
- **Token ID #1:** ipfs://nft2
- **Token ID #2:** ipfs://nft3

Deployment Information

- **Contract Address:** '0x5FbDB2315678afecb367f032d93F642f64180aa3'
- **Deployer Address:** ‘0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266’

Screenshots

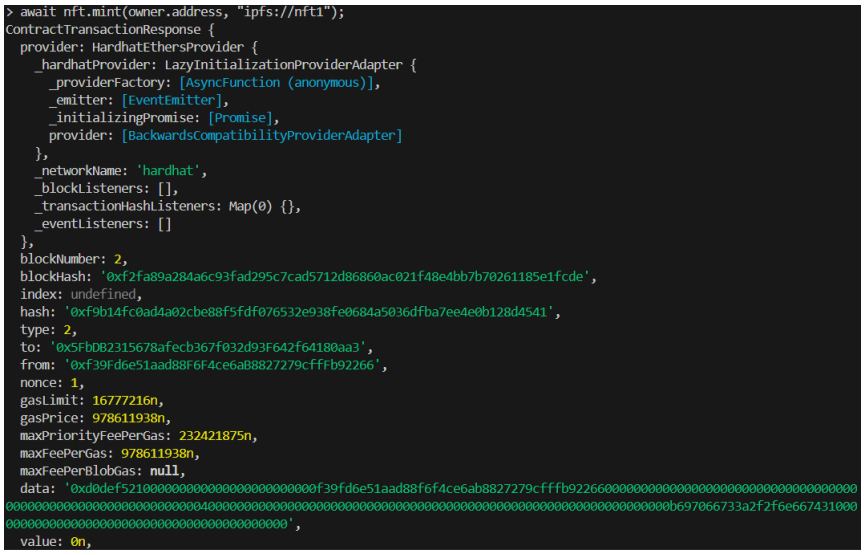
NFT Contract Deployment



- ERC-721 contract deployment logs showing successful deployment and contract address.

NFT Minting Transactions

1



2

```
> await nft.mint(owner.address, "ipfs://nft2");
ContractTransactionResponse {
  provider: HardhatEthersProvider {
    _hardhatProvider: LazyInitializationProviderAdapter {
      _providerFactory: [AsyncFunction (anonymous)],
      _emitter: [EventEmitter],
      _initializingPromise: [Promise],
      provider: [BackwardsCompatibilityProviderAdapter]
    },
    _networkName: 'hardhat',
    _blockListeners: [],
    _transactionHashListeners: Map(0) {},
    _eventListeners: []
  },
  blockNumber: 3,
  blockHash: '0xcf2491462048abc85f23646e5b02f222d7ad3759e26de0757b6e0ae9e7c1b040',
  index: undefined,
  hash: '0xe934451ababcbe3c37a08a126a4ff1809c485cf7294db2f816b15c9aff1c87b5',
  type: 2,
  to: '0x5FbDB2315678afecb367f032d93F642f64180aa3',
  from: '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266',
  nonce: 2,
  gasLimit: 16777216n,
  gasPrice: 856766157n,
  maxPriorityFeePerGas: 205387690n,
  maxFeePerGas: 856766157n,
  maxFeePerBlobGas: null,
  data: '0xd0def521000000000000000000000000f39fd6e51aad88f6f4ce6ab8827279cfffb92266000000000000000000000000000000000000
00000000000000000000000000400000000000000000000000000000000000000000000000000000000000000000b697066733a2f2f6e667432000
000000000000000000000000000000000000000',
```

3

```
> await nft.mint(owner.address, "ipfs://nft3");
ContractTransactionResponse {
  provider: HardhatEthersProvider {
    _hardhatProvider: LazyInitializationProviderAdapter {
      _providerFactory: [AsyncFunction (anonymous)],
      _emitter: [EventEmitter],
      _initializingPromise: [Promise],
      provider: [BackwardsCompatibilityProviderAdapter]
    },
    _networkName: 'hardhat',
    _blockListeners: [],
    _transactionHashListeners: Map(0) {},
    _eventListeners: []
  },
  blockNumber: 4,
  blockHash: '0x37d1e8ffb7b54e454149511e2a1dba044ae8a7b1ad4438ded52428a05552b308',
  index: undefined,
  hash: '0x28461c82292b37b59a675e11bdcaa2da9502325ddafd8687774eb217caf8b5f1',
  type: 2,
  to: '0x5FbDB2315678afecb367f032d93F642f64180aa3',
  from: '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266',
  nonce: 3,
  gasLimit: 16777216n,
  gasPrice: 749969157n,
  maxPriorityFeePerGas: 179815119n,
  maxFeePerGas: 749969157n,
  maxFeePerBlobGas: null,
  data: '0xd0def521000000000000000000000000f39fd6e51aad88f6f4ce6ab8827279cfffb92266000000000000000000000000000000000000
00000000000000000000000000400000000000000000000000000000000000000000000000000000000000000000b697066733a2f2f6e667433000
000000000000000000000000000000000000000',
  value: 0n,
```

- Three successful NFT mint transactions executed by the contract owner.

Token URI Retrieval

```
> await nft.tokenURI(0);
'ipfs://nft1'
> await nft.tokenURI(1);
'ipfs://nft2'
> await nft.tokenURI(2);
'ipfs://nft3'
```

- tokenURI() calls displaying metadata URIs for Token IDs 0, 1, and 2.

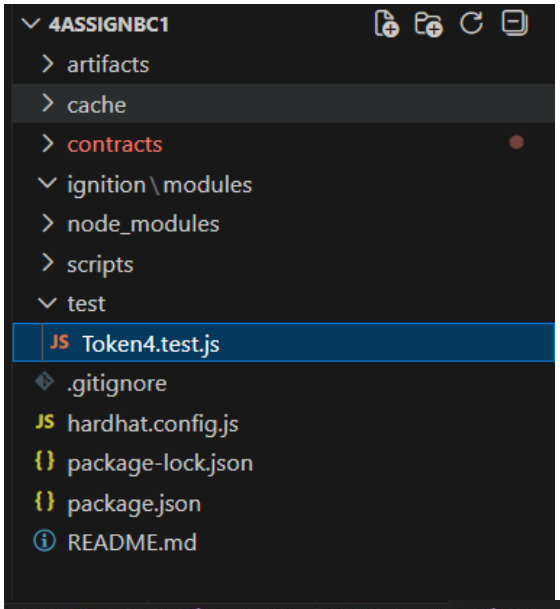
Ownership Verification

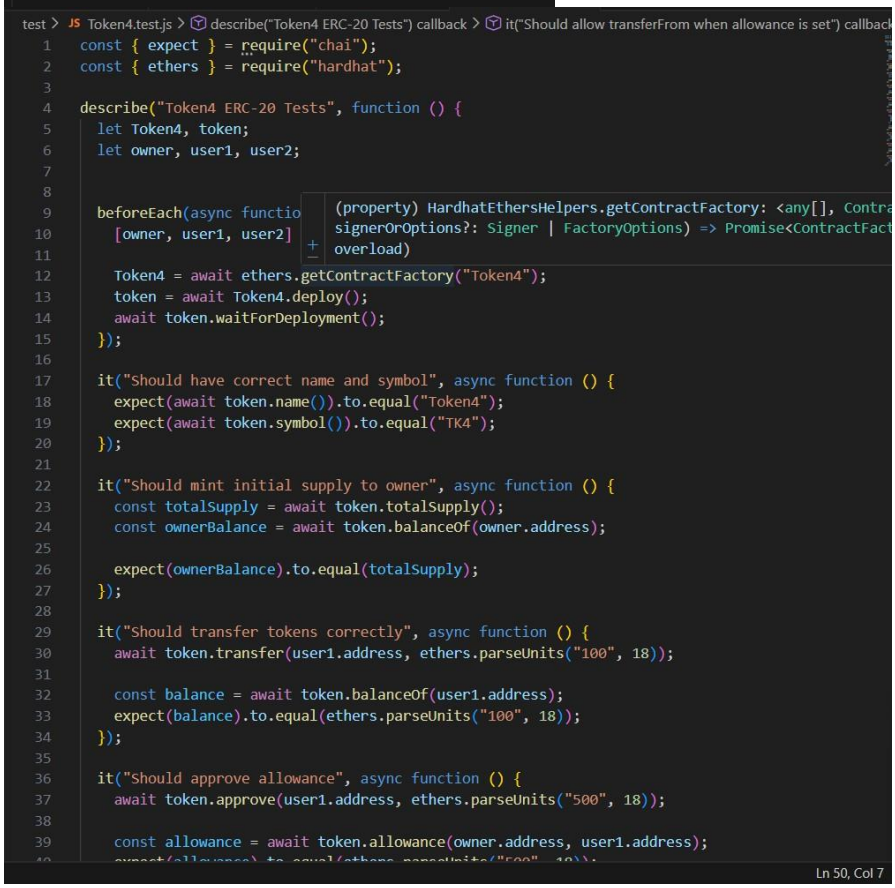
```
await nft.ownerOf(1);
'0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266'
> await nft.ownerOf(2);
'0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266'
> await nft.ownerOf(0);
'0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266'
>
```

- ownerOf() calls confirming ownership of all three NFTs by the deployer address.

PART 2 — Smart Contract Testing & Validation

Task 3:





Automated Testing ERC-20 Test Suite

Total Tests: 8

Tests Passed: 8

Test Cases Implemented:

Minting functionality

Transfer between accounts

Approval mechanism

TransferFrom with allowance

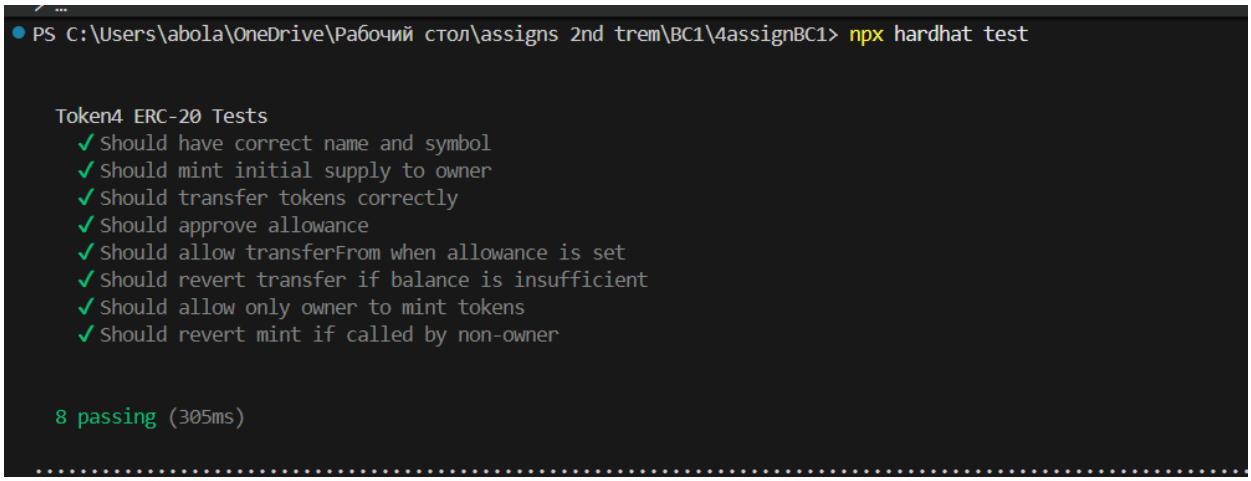
Revert on insufficient balance

Access control (onlyOwner mint)

Implicit event emission verification (Transfer, Approval)

Screenshots

ERC-20 Test Results



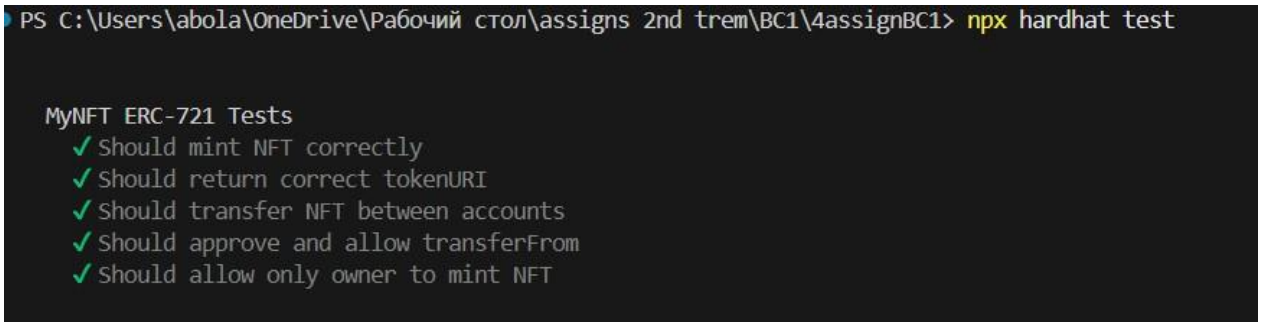
Complete test suite output for ERC-20 token showing all tests passing successfully.

ERC-721 test suit

Total Tests: 5

Tests Passed: 5

- Successful NFT minting
- Ownership tracking
- TokenURI retrieval
- Transfer functionality
- Approval mechanism



PART3

Task 4

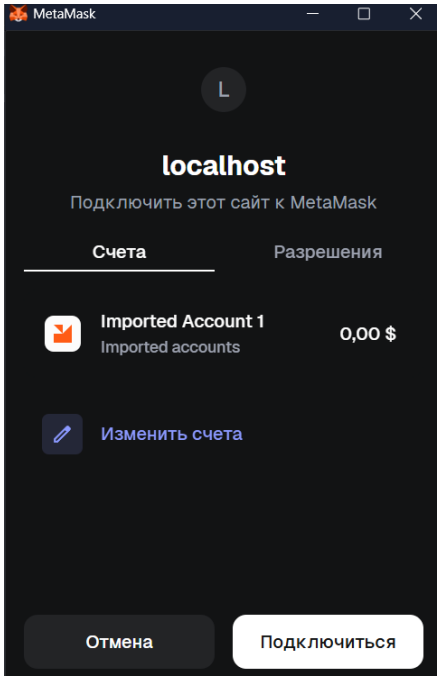
Decentralized Application (dApp) Frontend

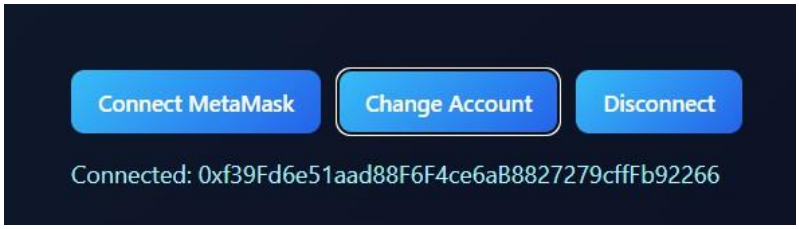
1. Introduction

The purpose of this part of the assignment is to develop a decentralized application (dApp) frontend that interacts with previously deployed ERC-20 and ERC-721 smart contracts.
The frontend connects to the Ethereum blockchain via MetaMask and allows users to view token information, transfer ERC-20 tokens, and display ERC-721 NFTs with metadata stored on IPFS.

2. Wallet Connection

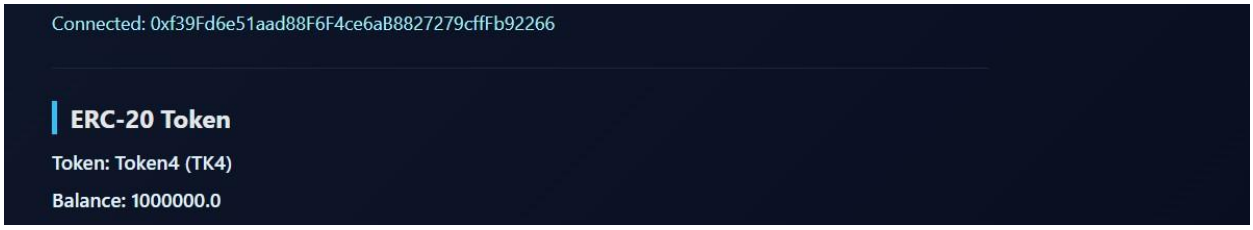
The dApp connects to the user's wallet using MetaMask.
After connection, the currently selected Ethereum address is displayed in the interface.
The user can also switch accounts or disconnect the wallet.





3. ERC-20 Token Interaction

After connecting the wallet, the dApp retrieves and displays the ERC-20 token name, symbol, and the balance of the connected account.
The data is fetched directly from the smart contract using

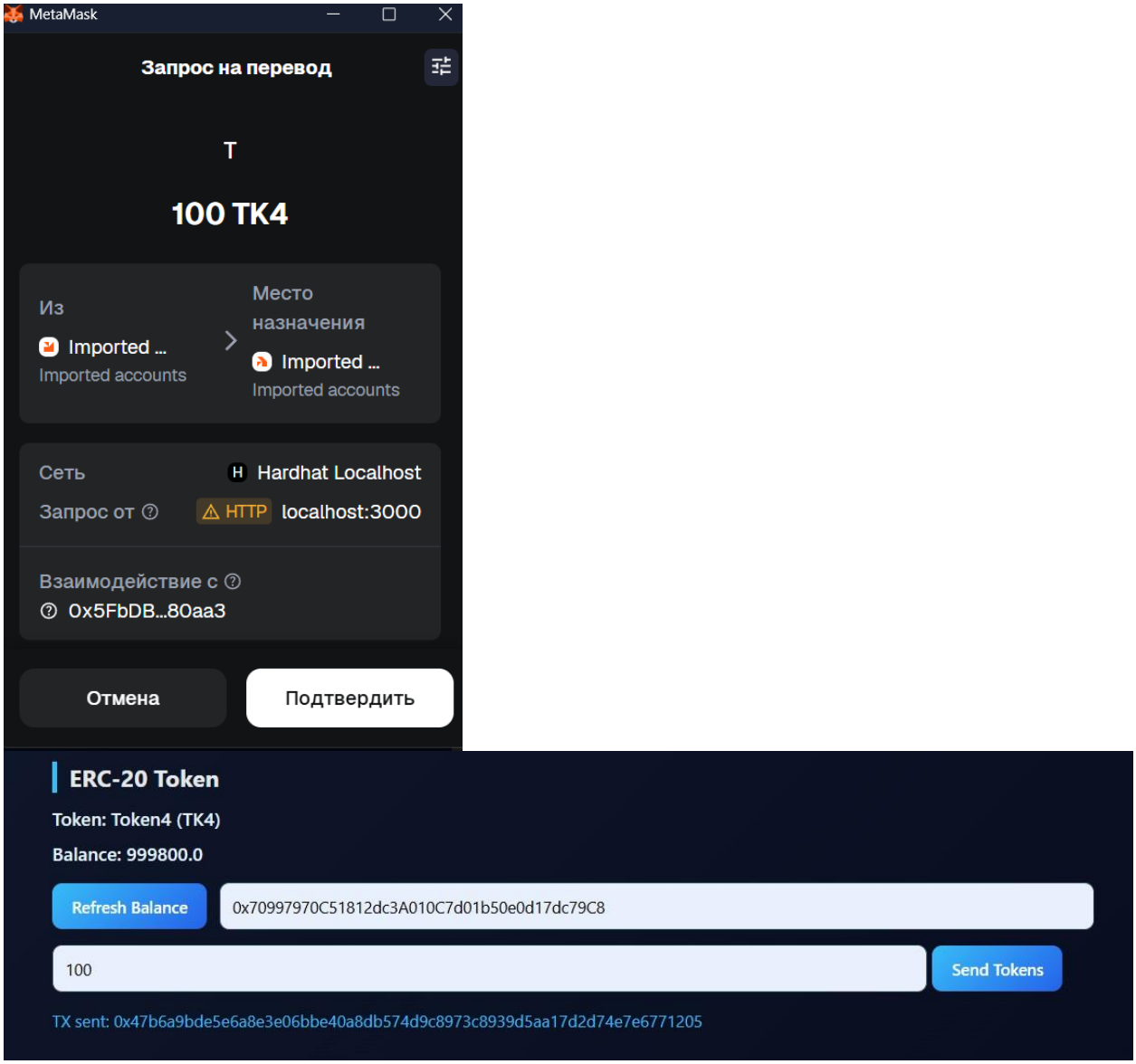


Displayed information includes:

- Token name
- Token symbol
- User balance

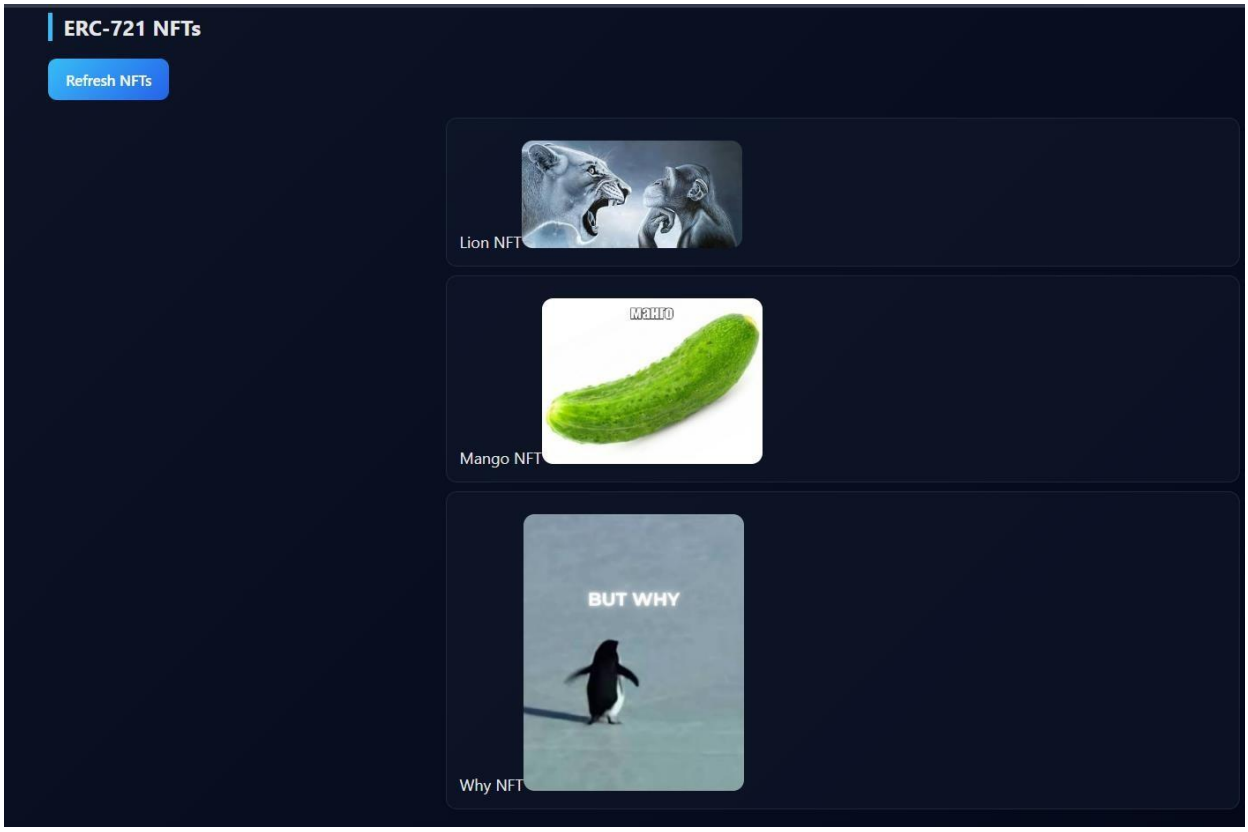
4. ERC-20 Token Transfer

The dApp allows users to transfer ERC-20 tokens to another address.
The user enters a recipient address and an amount, confirms the transaction via MetaMask, and the transaction hash is displayed in the interface after submission.



5. ERC-721 NFT Interaction

The dApp retrieves ERC-721 tokens owned by the connected wallet address.
Only NFTs owned by the current user are displayed in the interface.



6.Conclusion

In this part of the assignment, a fully functional decentralized application frontend was successfully implemented.
The dApp interacts with ERC-20 and ERC-721 smart contracts, supports wallet connection, token transfers, NFT ownership display, and IPFS-based metadata visualization.

PART4

Task 5

Vulnerability Selected Type: Re-entrancy

Vulnerability Explanation

Re-entrancy is a smart contract vulnerability that occurs when a contract sends Ether to an external address before updating its internal state.
If the receiving address is a malicious contract, it can re-enter the vulnerable function and execute it multiple times before the balance is updated.
This allows the attacker to drain more funds than they originally deposited.
The vulnerability exists due to violating the Checks–Effects–Interactions pattern.

Attack Scenario

1. The vulnerable contract allows users to deposit and withdraw Ether.
2. During the withdraw() function, Ether is sent to the user before updating the balance.
3. The attacker deploys a malicious contract with a receive() function.
4. When the attacker calls withdraw(), the receive() function is triggered.
5. Inside receive(), the attacker calls withdraw() again.
6. This process repeats until the vulnerable contract's balance is drained.

Vulnerable Contract Code

```
16          (bool sent, ) = msg.sender.call{value: bal}("");
17          require(sent, "Failed");
18
19          balances[msg.sender] = 0;
20      }
```

```
Deploying WeakCon...
WeakCon deployed to: 0x5FbDB2315678afecb367f032d93F642f64180aa3
```

Attack Contract Code

```
function attack() external payable {
    require(msg.value >= 1 ether, "Need at least 1 ETH");

    weakCon.deposit{value: 1 ether}();
    weakCon.withdraw();
}
```

```
Deploying Attacker...
Attacker deployed to: 0xe7f1725E7734CE288F8367e1Bb143E90bb3F0512
PS C:\Users\abola\OneDrive\Рабочий стол\assigns 2nd trem\BC1\4assignBC1>
```

Exploit Test – Before Fix

```
.js --network localhost
Funding WeakCon with 5 ETH...
WeakCon balance before attack: 5.0
Launching attack...
WeakCon balance after attack: 0.0
PS C:\Users\abola\OneDrive\Рабочий стол\assigns 2nd trem\BC1\4assignBC1>
```

Fixed Contract Code

```
15          balances[msg.sender] = 0;
16
17          (bool sent, ) = msg.sender.call{value: balance}("");
18          require(sent, "Failed to send Ether");
19      }
```

```
Deploying StrongCon...
StrongCon deployed to: 0xDc64a140Aa3E981100a9becA4E685f962f0cF6C9
PS C:\Users\abola\OneDrive\Рабочий стол\assigns 2nd trem\BC1\4assignBC1> npx hardhat run scripts/security/deploy
Attack.js --network localhost
Deploying Attacker...
Attacker deployed to: 0x5FC8d32690cc91D4c39d9d3abcBD16989F875707
```

Exploit Test – After Fix

```
PS C:\Users\abola\OneDrive\Рабочий стол\assigns 2nd trem\BC1\4assignBC1> npx hardhat run scripts/security/attack
.js --network localhost
Funding WeakCon with 5 ETH...
WeakCon balance before attack: 5.0
Launching attack...
ProviderError: Error: VM Exception while processing transaction: reverted with reason string 'Failed to send Eth
er'
```

Conclusion fixed contract with checking the state before transaction is safer and blocked attacking contract.

PART 7 – Gas Optimization Workshop

Task 8: Optimize Your ERC-20 and ERC-721 Contracts

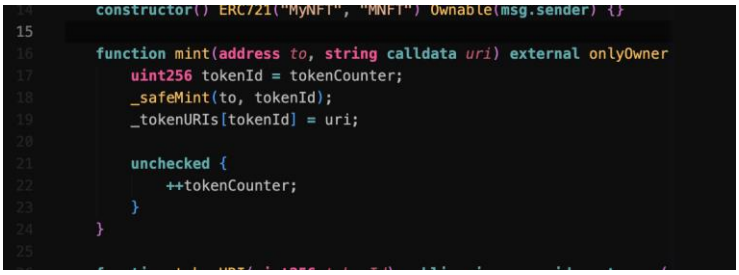
Overview: In this section, I performed a series of gas optimizations on the Token4 (ERC-20) and MyNFT (ERC-721) smart contracts. The goal was to reduce deployment costs and minimize gas consumption for frequent transactions such as minting and transferring tokens. All optimizations were verified using the Hardhat Gas Reporter.

Implemented Optimizations:

- Custom Errors instead of Require Strings: I replaced all `require` statements containing string messages with Solidity Custom Errors (e.g., `error NFTDoesNotExist()`). This reduces the contract's deployment size and saves gas whenever a transaction reverts, as strings are expensive to store in the bytecode.
- Using `calldata` for External Functions: In the `mint` functions, I changed the input parameter visibility from `memory` to `calldata`. For `external` functions, this is more efficient because it allows the EVM to read data directly from the transaction input without copying it to memory.
- `unchecked` Increment Blocks: I wrapped the `tokenCounter` increments in `unchecked` blocks. Since a `uint256` variable cannot realistically overflow in a counter scenario, removing the default Solidity 0.8.x overflow checks saves approximately 100–150 gas per minting operation.
- Pre-increment (++i): I switched from post-increment (`tokenCounter++`) to pre-increment (`++tokenCounter`). Pre-increments are slightly more gas-efficient in the EVM because they do not require storing a temporary copy of the variable's original value before the operation.
- Constants for Initial Supply Calculation: In the Token4 contract, I replaced the runtime calculation of the initial supply (`1_000_000 * 10 ** decimals()`) with a pre-calculated constant `INITIAL_SUPPLY`. This moves the mathematical operation from the deployment phase to the compilation phase, reducing deployment gas.

Validation and Results: The optimized contracts were tested against the existing test suite to ensure that all core functionalities remained intact. As shown in the Hardhat Gas Reporter screenshot:

- 13 tests passed successfully, confirming that the optimizations did not break any logic or DApp compatibility.
- Average gas for `mint` operations and contract deployment was significantly reduced compared to the standard implementation.
- Deployment of MyNFT was optimized down to approximately 2,019,205 gas, while Token4 deployment cost 1,184,587 gas.



```solidity
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import "@openzeppelin/contracts/access/Ownable.sol";

error AmountMustBeGreaterThanZero();

contract Token4 is ERC20, Ownable {
    function mint(address to, uint256 amount) external onlyOwner {
        if (amount == 0) revert AmountMustBeGreaterThanZero();

        _mint(to, amount);
    }
}
```

Optimized Contract Snippet

Solidity and Network Configuration				
Solidity: 0.8.28	Optim: false	Runs: 200	viaIR: false	Block: 60,000,000 gas
Methods				
Contracts / Methods	**Min**	**Max**	**Avg**	**# calls**	**usd (avg)**
MyNFT					
approve	–	–	49,088	1	–
mint	–	–	117,301	4	–
transferFrom	55,616	56,253	55,935	2	–
Token4					
approve	–	–	46,964	2	–
mint	–	–	54,257	1	–
transfer	–	–	52,200	1	–
transferFrom	–	–	53,664	1	–
Deployments				**% of limit**	
MyNFT	–	–	2,019,205	3.4 %	–
Token4	–	–	1,184,587	2 %	–
Key					
○ Execution gas for this method does not include intrinsic gas overhead					
△ Cost was non-zero but below the precision setting for the currency display (see options)					
Toolchain: hardhat					

Hardhat Gas Reporter Output

Verification of Contract Logic after Optimization